

AI TRADING SYSTEM

Full-Stack Autonomous Stock Trading Platform

NSE / NIFTY50 | Machine Learning | Paper Trading | Cloud Deployment



What Was Built

A complete, production-grade autonomous stock trading system for the Indian NSE/NIFTY50 market. The system accepts only two inputs from the user - Total Capital and Risk Profile - and handles everything else automatically: stock scanning, capital allocation, AI-driven trade signals, risk validation, paper order execution, portfolio rebalancing, model retraining, and daily/weekly/monthly reporting.

The entire system runs in `SIMULATION_MODE=True` by default. No real broker orders are ever placed unless explicitly enabled. Every component is covered by automated tests.

12-Phase Development Journey

Phase	Description
Phase 1-2 Data Infrastructure	Yahoo Finance collector, SQLite DB, OHLCV validator, live quotes
Phase 3 News & Sentiment Engine	Multi-source news (RSS/AV/Marketaux), VADER sentiment, SQLite store
Phase 4 AI Feature Engineering	20+ technical indicators: RSI, MACD, EMA, ATR, Bollinger, volume ratios
Phase 5 Professional Backtesting	Bar-by-bar backtest, 5 strategies, walk-forward, Monte Carlo, portfolio BT
Phase 6 Paper Trading System	Virtual broker, order lifecycle, portfolio P&L, slippage & fees model
Phase 7 Risk Management Engine	Position sizing (ATR/Kelly/Fixed%), 10-gate trade validator, portfolio risk
Phase 8 Broker Integration	Adapters: Zerodha, Dhan, AngelOne, Groww - all default <code>SIMULATION_MODE</code>
Phase 9 Live Execution Engine	Real-time loop, stop-loss/take-profit/trailing/EOD exits, trade journal
Phase 10 Cloud Deployment	Docker, docker-compose, VPS Supervisor, Telegram/Email/Discord alerts
Phase 11 Advanced AI Optimisation	Ensemble RF+XGB, market regime, drift detection (PSI), auto-retraining
Phase 12 Fully Autonomous Mode	Stock scanner, capital allocator, safety system, rebalancer, reporter
Audit Production Hardening	Security fixes, disaster recovery, E2E simulation, validation report

System Architecture

The system is organised into layered packages. Each layer depends only on layers below it, making individual components testable in isolation.

Layer	Components / Responsibility
Dashboard (Streamlit)	12 pages: Overview, News, Sentiment, Stock Analysis, AI Models, Predictions, Backtesting, Paper Trading, Risk, Broker Status, Live Trading, AI Analytics, System Health, Autonomous Trading
Autonomous Engine	AutonomousEngine orchestrator: scan -> regime -> allocate -> execute -> monitor -> rebalance -> model manage -> report
Execution Layer	LiveExecutionEngine, TradeExecutor, TradeMonitor, TradeJournal, OrderManager - NSE market hours guard, simulation-first
AI / ML Layer	EnsembleModel (RF+XGB), PredictionEngine, FeatureEngineeringService, LabelGenerator, MarketRegimeDetector, DriftDetector, AutoRetrainer
Risk Management	RiskEngine -> PositionSizer + TradeValidator + PortfolioRiskAnalyzer All 10 pre-trade gates enforced before any order
Paper Trading	PaperBroker -> PaperPortfolio + PaperOrderManager Full order lifecycle (PENDING -> FILLED/REJECTED/CANCELLED)
Broker Adapters	ZerodhaAdapter, DhanAdapter, AngelOneAdapter, GrowwAdapter All raise NotImplementedError if live mode attempted without explicit flag
Data Collection	YahooFinanceProvider, HistoricalDataCollector, LiveDataCollector OHLCVValidator, SQLite repositories
News & Sentiment	NewsCollector (4 providers), SentimentEngine (VADER), NewsRepository, SentimentRepository
Monitoring & Recovery	SystemMonitor (psutil), DisasterRecovery, BackupManager, TelegramAlerter, EmailAlerter, DiscordAlerter

Folder Structure (Key Paths)

Path	Contents
ai_trading_system/	Core settings, DB connection, data collection, validators
features/	FeatureEngineeringService, LabelGenerator, 20+ indicators
models/	RandomForestModel, XGBoostModel, ModelRegistry, BaseModel
prediction/	PredictionEngine with full error-handling + HOLD fallback
risk_management/	RiskEngine, PositionSizer, TradeValidator, PortfolioRiskAnalyzer
paper_trading/	PaperBroker, PaperPortfolio, PaperOrderManager, PaperReports
broker/	BaseBroker + 4 live adapters (all simulation-safe)
execution/	LiveExecutionEngine, TradeExecutor, TradeMonitor, OrderManager
advanced_ai/	EnsembleModel, MarketRegimeDetector, DriftDetector, AutoRetrainer
autonomous/	AutonomousEngine, StockScanner, CapitalAllocator, SafetySystem, PortfolioRebalancer, Reporter, ModelManager, RiskProfiles
monitoring/	SystemMonitor, DisasterRecovery, BackupManager
alerts/	TelegramAlerter, EmailAlerter, DiscordAlerter
dashboard/	streamlit_app.py - all 14 pages
tests/	340 tests across 9 test files - 100% pass rate
data/historical/	CSV files - SYMBOL_NS.csv (e.g. RELIANCE_NS.csv)
backups/	Timestamped backup sets: DB + models + trade journal

Fully Autonomous Trading - Phase 12

The user provides only two values. The system handles everything else automatically, 24/7, with no manual intervention required.

Input 1: Total Capital

e.g. Rs 50,000

Input 2: Risk Profile

conservative / moderate / aggressive

Autonomous Pipeline (per tick, every 5 minutes during market hours)

- Market hours check (NSE 9:15 AM - 3:30 PM IST, holidays excluded)
- Safety system check - daily loss / confidence collapse / broker failure / market anomaly
- Market regime detection - BULL / BEAR / SIDEWAYS / HIGH_VOL / LOW_VOL (EMA + ADX + volatility)
- Stock scan - NIFTY50 universe, composite score: Momentum 35% + Volume 20% + Sentiment 20% + AI 25%
- Capital allocation - inverse-volatility weighting, cash reserve, stop/target via ATR x profile multiplier
- Trade execution - paper broker (simulation), records to TradeJournal CSV
- Position monitoring - stop-loss / take-profit / trailing stop / time exit / EOD exit / AI exit
- Portfolio rebalancing - drift detection, regime-based cash target, concentration cap
- Model management - drift detection (PSI), accuracy tracking, auto-retrain on trigger
- Reporting - daily/weekly/monthly via Telegram + Email, automatic scheduling

Risk Profiles

Profile	Positions	Risk/Trade	Min AI Confidence	Cash Reserve
conservative	3 positions max	0.5% risk/trade	70% min AI confidence	30% cash reserve
moderate	5 positions max	1.5% risk/trade	62% min AI confidence	15% cash reserve
aggressive	8 positions max	3.0% risk/trade	55% min AI confidence	5% cash reserve

Safety System - Emergency Halts

Halt Type	Condition & Behaviour
Daily Loss Limit	Halts if today's P&L < -(capital x daily_loss_limit_pct). Resets at market open next day.
Confidence Collapse	Halts if rolling avg AI confidence (last 10 predictions) falls below 45%. Signals model degradation.
Broker Failure	Halts immediately if broker snapshot() fails. Prevents ghost trades with stale state.
Market Anomaly	Halts if latest bar return z-score > 4.0 (extreme 4-sigma spike). Protects against data errors or flash crashes.
Manual Stop	User-triggered emergency halt via dashboard button or safety.manual_stop(). Resume via reset_safety().

Test Suite - 340 Tests, 0 Failures

Test File	Coverage
test_phase9_execution.py	21 tests - TradeExecutor, TradeMonitor, LiveExecutionEngine, MarketHours
test_phase10_monitoring.py	18 tests - SystemMonitor, TelegramAlerter, EmailAlerter, DiscordAlerter
test_phase11_ai.py	40 tests - EnsembleModel, MarketRegime, DriftDetector, AutoRetrainer
test_phase12_autonomous.py	48 tests - RiskProfiles, StockScanner, CapitalAllocator, SafetySystem, Rebalancer, Reporter, Engine
test_e2e_simulation.py	57 tests - Full pipeline: Data -> Features -> AI -> Predict -> Risk -> Trade -> DR
test_phase5_backtest.py	~35 tests - BacktestEngine, 5 strategies, WalkForward, MonteCarlo
test_phase6_paper.py	~30 tests - PaperBroker, PaperPortfolio, PaperOrderManager
test_phase7_risk.py	~40 tests - PositionSizer, TradeValidator, PortfolioRiskAnalyzer
test_phase8_broker.py	~51 tests - OrderManager, ExecutionEngine, all broker adapters

Production Audit Results

Critical Issues Found & Fixed

SQL Injection (dashboard/streamlit_app.py:29)

Issue: Table name interpolated into SQL string

Fix: Added `_ALLOWED_TABLES` whitelist - rejects any unrecognised table name

Prediction Crash (prediction/prediction_engine.py:38)

Issue: No validation - empty DF / wrong features / bad proba -> `IndexError`

Fix: Full guard: empty check -> model fallback -> feature alignment -> exception catch -> HOLD sentinel

High Priority Issues Fixed

Session State Race (dashboard/streamlit_app.py:429)

Replaced `session_state` dict with `@st.cache_resource` singleton - eliminates dual-broker race condition

Unsafe HTML (dashboard/streamlit_app.py:52)

Clamped interval to `max(10,min(300,int(x)))` before inserting into meta refresh tag

Verified GOOD - No Action Required

- All SQLite queries parameterized with `?` placeholders throughout
- No hardcoded secrets - all API keys, tokens, passwords via `.env` / environment variables
- All network calls have retry, timeout, and exception handling
- Risk engine enforces 10 pre-trade gates before any order reaches the broker
- Paper broker validates cash, position existence, and trading-enabled flag on every order
- Database connections always closed via `@contextmanager` - no connection leaks

Disaster Recovery System

Full backup-and-restore lifecycle implemented in `monitoring/disaster_recovery.py`

Method	Action
<code>backup_all()</code>	Hot-backup DB via <code>sqlite3.backup()</code> API + copy models + trade journal + config
<code>restore_latest()</code>	Auto-finds newest backup, verifies integrity, restores all components
<code>restore(backup_id)</code>	Restore specific backup set by timestamp ID
<code>restore(dry_run=True)</code>	Preview what would be restored - no files written
<code>recover_trade_logs()</code>	Merge all backup CSVs, deduplicate on <code>trade_id</code> , write recovered journal
<code>server_recovery()</code>	7-step cold-start: find -> verify -> restore DB -> restore models -> restore log -> rebuild -> init schema
<code>verify_backup(id)</code>	Run <code>PRAGMA integrity_check</code> on DB backup + check all file flags
<code>list_backups()</code>	Return all backup manifests newest-first for retention management

Historical Data - Exact Path & Download

Both the Dashboard Stock Analysis page and the Autonomous StockScanner use identical paths:

Folder: `data/historical/`

File: `{SYMBOL}.replace('.', '_') + '.csv'`

Example: `RELIANCE.NS -> data/historical/RELIANCE_NS.csv`

Download: `python main.py --collect-history --symbol RELIANCE.NS --years 2`

Quick-Start Commands

Task	Command
Initialise DB	<code>python main.py --init-db</code>
Download data	<code>python main.py --collect-history --symbol RELIANCE.NS --years 2</code>
Train models	<code>python main.py --train-models --data-file data/historical/RELIANCE_NS.csv</code>
Run backtest	<code>python main.py --backtest --data-file data/historical/RELIANCE_NS.csv</code>
Paper trade (1 tick)	<code>python main.py --paper-trade</code>
Live engine (sim)	<code>python main.py --live-engine --ticks 5</code>

Autonomous mode

	<code>python main.py --autonomous --capital 50000 --risk-profile moderate</code>
Launch dashboard	<code>streamlit run dashboard/streamlit_app.py</code>
Docker deploy	<code>docker-compose up -d</code>
Backup system	<code>python main.py --backup</code>
Run all tests	<code>python -m pytest -q (340 tests, ~42 s)</code>
E2E validation	<code>python -m pytest tests/test_e2e_simulation.py -s -q</code>

PRODUCTION VALIDATION REPORT

Score: 100.0% | Status: PRODUCTION READY

Total Tests

340

Passed

340

Failed

0

Warnings

0

End-to-End Pipeline Results

Stage	Result	Key Validations
Stage 1: Data Validation	100%	OHLCVValidator accepts clean data, rejects empty/corrupt inputs
Stage 2: Feature Engineering	100%	20+ indicators, no inf/NaN columns, stable output across runs
Stage 3: AI Model Training	100%	RF and XGBoost train, save, load, predict_proba correctly
Stage 4: Prediction Engine	100%	Graceful HOLD on empty input / no model / wrong features
Stage 5: Risk Engine	100%	All 10 validation gates enforced; zero-price and low-confidence blocked
Stage 6: Paper Trading	100%	Buy/sell lifecycle, P&L calculation, insufficient cash rejected
Stage 7: Disaster Recovery	100%	Backup -> integrity check -> dry-run restore -> trade log merge

Technology Stack

Component	Technology
Language	Python 3.11+
ML Models	scikit-learn (RandomForest), XGBoost, Optuna (hyperparameter search)
Data	yfinance, pandas, numpy - OHLCV + live quotes + news
NLP / Sentiment	VADER (vaderSentiment) - news sentiment scoring
Dashboard	Streamlit - 14-page interactive web UI
Database	SQLite3 - market data, news, sentiment, predictions
Alerts	Telegram Bot API, SMTP Email, Discord Webhooks (urllib only - no requests dep)
Deployment	Docker + docker-compose, Supervisor (VPS), systemd-compatible
Testing	pytest - 340 tests, 100% pass rate
Monitoring	psutil (system health), rotating log handlers, backup manager

Key Design Decisions

- Simulation-first: SIMULATION_MODE=True on every broker adapter - real orders require explicit opt-in
- Graceful degradation everywhere: no model -> HOLD, no sentiment DB -> neutral 50%, no psutil -> skip metrics
- Frozen risk profiles (Python dataclasses) - immutable at runtime, preventing accidental mutation
- PSI (Population Stability Index) for drift detection - implemented with numpy only, no scipy dependency
- Hot SQLite backup via sqlite3.backup() API - safe to run while trading engine is active
- Bar-by-bar backtest execution - no look-ahead bias, each bar only sees past data
- 10-gate pre-trade validation - price, quantity, confidence, trend, positions, exposure, drawdown, cash
- All secrets via environment variables / .env - no hardcoded credentials anywhere in codebase

